

FIAP

Engenharia de Software

Computational Thinking with Python

Câmera Inteligente com OCR

Sprint 2 – Documentação Técnica

Integrantes do Grupo:

Artur Fabi Brandi – RM570258

Felipe Santos Ribas – RM569121

Luiz Gonzaga – RM572446

Orion Cavalcante França – RM573677

Victor Lula Heineken Rodrigues – RM570782

São Paulo, 2025

SUMÁRIO

1	Descritivo do Projeto	3
	1.1 Contexto e Motivação	3
	1.2 Objetivo Geral	3
	1.3 Justificativa	3
	1.4 Público-Alvo	3
2	Descrição das Funcionalidades Implementadas	4
	2.1 Simular Captura de Texto (OCR)	4
	2.2 Pesquisar Texto no Google	4
	2.3 Traduzir Texto Capturado	4
	2.4 Visualizar Histórico de Capturas	5
	2.5 Limpar Histórico	5
3	Conceitos de Python Aplicados	5
4	Estrutura do Código-Fonte	6

1. DESCRITIVO DO PROJETO

1.1 Contexto e Motivação

O fluxo de pesquisa acadêmica envolve etapas repetitivas: localizar um texto (em livro, lousa, slide ou documento), digitá-lo manualmente em um buscador ou tradutor, e só então obter a informação desejada. Esse processo gera fricção desnecessária e dispersa o foco do estudante.

A feature proposta elimina essas etapas ao integrar a câmera do smartphone com ferramentas de pesquisa e tradução, utilizando OCR (Reconhecimento Óptico de Caracteres) para capturar textos do ambiente e convertê-los em ações imediatas.

1.2 Objetivo Geral

Desenvolver, em Python, uma simulação do fluxo de uma câmera inteligente com OCR: captura de texto, armazenamento em histórico, pesquisa no Google e tradução — reduzindo etapas manuais no processo de pesquisa acadêmica.

1.3 Justificativa

Os cenários de uso contemplados são diretamente ligados ao cotidiano universitário:

- Capturar conteúdo de lousa e pesquisá-lo sem trocar de aplicativo;
- Digitalizar trechos de livros em língua estrangeira e traduzir sem copiar manualmente;
- Registrar anotações manuscritas de aula para referência posterior;
- Consultar slides projetados em sala de forma rápida.

O diferencial da solução é transformar a câmera de ferramenta passiva de captura em ferramenta ativa de pesquisa e aprendizado.

1.4 Público-Alvo

Estudantes universitários em aulas presenciais ou híbridas que precisam pesquisar, traduzir ou registrar conteúdos de forma rápida durante o processo de estudo.

2. DESCRIÇÃO DAS FUNCIONALIDADES IMPLEMENTADAS

2.1 Simular Captura de Texto (OCR)

Simula o processo de detecção automática de texto que, em um aplicativo real, seria executado pelo motor OCR ao apontar a câmera para uma superfície. O usuário insere o texto detectado e informa o tipo de fonte.

Regras de negócio:

- Textos em branco ou com menos de 3 caracteres são rejeitados;
- A classificação do tipo de fonte (impresso, lousa/slide, manuscrito) é obrigatória e validada em loop;
- Cada captura é armazenada como dicionário na lista `historico_capturas`.

2.2 Pesquisar Texto Capturado no Google

Gera a URL de pesquisa no Google para um texto previamente capturado, simulando a ação contextual que apareceria na interface da câmera após detecção.

Regras de negócio:

- Disponível apenas se houver ao menos uma captura no histórico;
- O usuário seleciona qual captura pesquisar; a entrada é validada em loop;
- O texto é codificado via `urllib.parse.quote` para compor a URL de forma segura.

2.3 Traduzir Texto Capturado

Gera um link direto para o Google Translate com o texto e o idioma de destino já configurados, eliminando a necessidade de copiar e colar manualmente.

Regras de negócio:

- Disponível apenas se houver capturas no histórico;
- Idiomas suportados: Inglês, Espanhol, Francês e Alemão;
- Seleção de texto e de idioma são validadas em loop até entrada correta;
- A URL é gerada com o código ISO 639-1 do idioma via f-string.

2.4 Visualizar Histórico de Capturas

Exibe todas as capturas registradas na sessão, com índice, tipo de fonte e texto completo, permitindo ao usuário revisar o conteúdo detectado anteriormente.

Regras de negócio:

- Se o histórico estiver vazio, uma mensagem informativa é exibida;
- A listagem percorre a lista com `for/enumerate`, formatando cada item com f-string;

- O total de registros é exibido antes da listagem.

2.5 Limpar Histórico

Remove todos os registros da lista de capturas após confirmação explícita, prevenindo apagamentos acidentais.

Regras de negócio:

- Se o histórico estiver vazio, a operação é informada sem solicitar confirmação;
- O total de itens a remover é exibido na pergunta de confirmação;
- Apenas a resposta "s" confirma; qualquer outra entrada cancela a operação.

3. CONCEITOS DE PYTHON APLICADOS

Conceito	Aplicação no Projeto
Manipulação de variáveis	Texto capturado, tipo de fonte, índice de seleção, URL gerada
Entrada/saída com f-string	Todas as mensagens de feedback e URLs exibidas ao usuário
Validação de entrada	Loops que rejeitam texto vazio/curto, índice fora de intervalo e caractere não numérico
Estruturas de decisão	match-case no menu principal; if-elif-else nas validações internas
Estruturas de repetição	while True no menu e nas validações; for/enumerate na listagem
Listas	historico_capturas: lista de dicionários com append, clear e iteração
Organização do código	Funções por responsabilidade; selecionar_captura() reutilizada em duas funcionalidades
Comentários e usabilidade	Docstrings em todas as funções; prefixos [!] para erros e [✓] para sucesso

4. ESTRUTURA DO CÓDIGO-FONTE

O arquivo `projeto.py` é dividido em funções com responsabilidade única. A função auxiliar `selecionar_captura()` é reutilizada pelas funcionalidades de pesquisa e tradução, evitando duplicação de código.

Função	Responsabilidade
<code>exibir_menu()</code>	Renderiza o menu principal com todas as opções numeradas
<code>simular_captura_ocr()</code>	Coleta e valida texto e tipo de fonte; adiciona ao histórico
<code>selecionar_captura(acao)</code>	Auxiliar: lista capturas disponíveis e retorna a escolhida com validação
<code>pesquisar_no_google()</code>	Usa <code>selecionar_captura()</code> e gera URL de busca no Google
<code>traduzir_texto()</code>	Usa <code>selecionar_captura()</code> , coleta idioma e gera URL do Google Translate
<code>visualizar_historico()</code>	Itera sobre a lista e exibe todas as capturas formatadas
<code>limpar_historico()</code>	Solicita confirmação e executa <code>historico_capturas.clear()</code>
<code>main()</code>	Loop principal com <code>match-case</code> ; ponto de entrada do programa

Loop principal

```
def main():
    print("Bem-vindo ao Sistema de Câmera Inteligente com OCR!")

    while True:
        exibir_menu()
        opcao = input("Escolha uma opção: ").strip()

        match opcao:
            case "1":
                simular_captura_ocr()
            case "2":
                pesquisar_no_google()
            case "3":
                traduzir_texto()
            case "4":
                visualizar_historico()
            case "5":
                limpar_historico()
            case "0":
                print("Encerrando o sistema. Até logo!")
                break
            case _:
                print("[!] Opção inválida.")
```

Fluxo de execução

1. O programa inicia por `main()`, chamada via `if __name__ == "__main__"`.
2. O menu é exibido em loop até que o usuário selecione a opção **0 (Sair)**.
3. Cada opção executa a função correspondente e retorna ao loop principal.
4. O histórico persiste na lista global `historico_capturas` durante toda a sessão.